

School:Campus:

Academic Year: Subject Name: Subject Code:

Semester: Program: Branch: Specialization:

Date:

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiment: Contract QA – Testing Smart Contracts

*Aim :

To understand and perform quality assurance (QA) testing for smart contracts deployed on blockchain platforms such as Ethereum to ensure their correctness, reliability, and security.

*Objective:

- To learn how to test smart contracts using QA methodologies.
- To identify and resolve logical or security issues before deployment.
- To use simulation tools and test networks for contract verification.
- .

*Coding Phase: Pseudo Code / Flow Chart / Algorithm

1. Open Remix IDE and create a new Solidity smart contract (for example, SimpleStorage.sol).
2. Write a simple contract with basic functions such as store, retrieve, and modify.
3. Compile the smart contract to check for syntax or compilation errors.
4. Deploy the contract on a test network such as Sepolia.
5. Perform different QA test cases:
 - Positive testing: valid inputs and expected outcomes.
 - Negative testing: invalid inputs, edge cases, and gas limit checks.
 - Security testing: reentrancy, integer overflow or underflow, and access control.
6. Observe output logs and gas consumption during execution.
7. Analyze results and confirm whether the contract logic performs as intended.
8. Document any errors found and apply necessary fixes.
9. Re-test the contract until all test cases pass successfully.

Page No.....

*** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.**

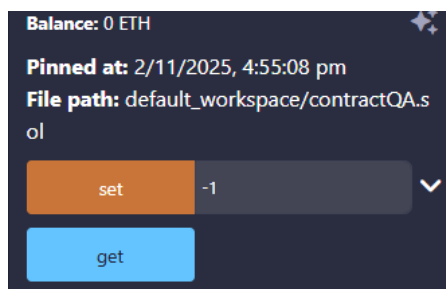
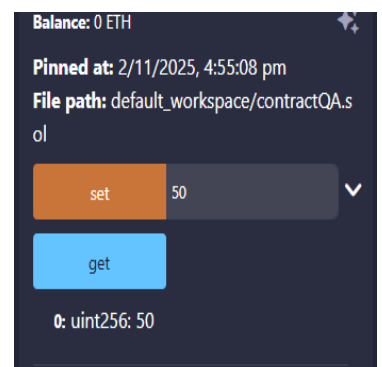
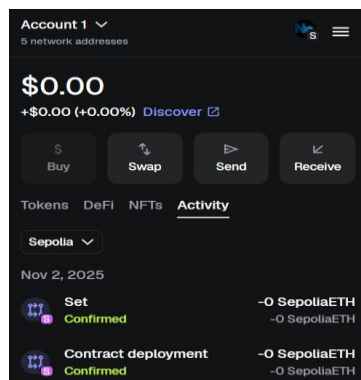
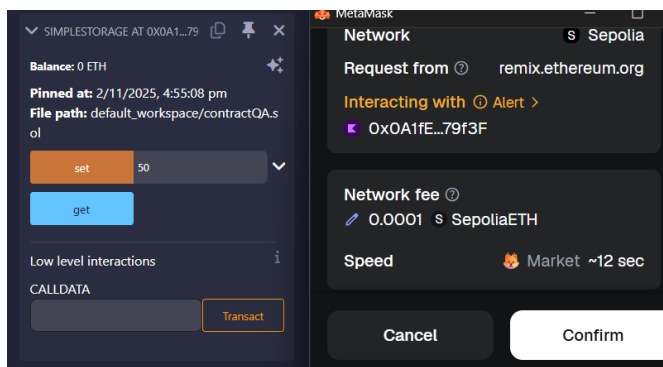
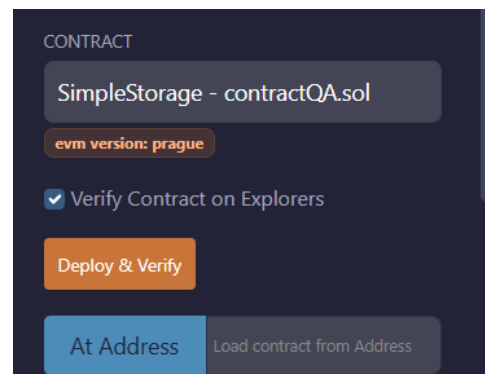
* Software used:

- Remix IDE
- Ethereum sepolia testnet
- Etherscan (testnet)

* Implementation Phase: Final Output (no error)

- Open the Remix IDE and write the smart contract.
- Compile the contract.
- Deploy using the “Injected Provider-MetaMask” environment.
- In the deployed contract tab:
- Use set() to store a value.
- Use get() to retrieve and verify.
- Confirm that the function outputs are correct.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract SimpleStorage {
5     uint256 private data;
6
7     function set(uint256 x) public { 22492 gas
8         data = x;
9     }
10
11     function get() public view returns (uint256) { 2431 gas
12         return data;
13     }
14 }
```



transact to SimpleStorage.set errored: Error encoding arguments: TypeError: value out-of-bounds (argument="", value="-1", code=INVALID_ARGUMENT, version=6.14.0)

** As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.*

*** Observation:**

- Smart contract testing ensures reliability and correctness before deployment.
- Logical or security bugs can be detected early through QA testing.
- Tools like Remix, MythX, and Etherscan help validate code and execution flow.

ASSESSMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	50		

Signature of the Student :

Name :

Signature of the Faculty :

Regn. No. :

Page No.....

** As applicable according to the experiment.*

Two sheets per experiment (10-20) to be used